🗓️ Planning d'implémentation — Assistant Matanne

> **Basé sur** : `ANALYSE_COMPLETE.md` (28 mars 2026)
> **Durée totale estimée** : 3+ mois
> **Légende efforts** : XS ≤ 1h | S = 2-4h | M = 1-2j | L = 3-5j

Vue d'ensemble des sprints

Sprint	Durée	Thème	Items
Sprint 1	Sem 1	Bugs critiques + SQL fondations	P-01 à P-07
Sprint 2	Sem 2	Bugs hauts + Nettoyage doc	B-06, B-07, B-11, B-12, CT-11, CT-15 à CT-17
Sprint 3	Sem 3	Tests manquants	CT-07, CT-12, CT-13, CT-14
Sprint 4	Sem 4	Features cuisine/famille/Jules	CT-05, CT-06, CT-09
Sprint 5	Sem 5-6	Notifications email + Admin étendu	CT-01, CT-02
Sprint 6	Sem 7-8	Cron jobs + SQL avancé	CT-03, CT-04, CT-08, CT-10
Sprint 7	Mois 2	Inter-modules + Dashboard	MT-01, MT-03, MT-06, MT-08, MT-09
Sprint 8	Mois 3	Canal WhatsApp + IA avancée	MT-02, MT-04~MT-07
Sprint 9	Mois 3+	Innovations & Long terme	LT-01 à LT-04 + Innovations

Sprint 1 — Bugs critiques + SQL fondations

> **Durée** : ~5 jours | **Priorité** : 🔴 BLOQUANT

Objectif

Corriger les bugs qui rendent des features entièrement inopérantes (push, notifications) et consolider la source de vérité SQL pour partir sur une base saine.

P-01 — Persistance push subscriptions en DB *(S)* {#p-01--persistance-push-subscriptions-en-db-*s* }

Bug : B-01 — `_subscriptions` dict en mémoire → perdu à chaque redémarrage

Fichiers à modifier/créer :

- `src/core/models/notifications.py` — ajouter modèle `AbonnementPushVapid`
- `src/services/core/notifications/notif_web_core.py` — charger/sauvegarder depuis DB
- `sql/INIT_COMPLET.sql` — ajouter bloc `CREATE TABLE abonnements_push_vapid`

SQL à ajouter dans INIT_COMPLET.sql :

```sql
CREATE TABLE IF NOT EXISTS abonnements_push_vapid (
    id BIGSERIAL PRIMARY KEY,
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
    endpoint TEXT UNIQUE NOT NULL,
    p256dh TEXT NOT NULL,
    auth_key TEXT NOT NULL,
    user_agent TEXT,
    actif BOOLEAN DEFAULT TRUE,
    cree_le TIMESTAMPTZ DEFAULT NOW(),
    dernier_ping TIMESTAMPTZ
);
ALTER TABLE abonnements_push_vapid ENABLE ROW LEVEL SECURITY;
CREATE POLICY "push_vapid_user_policy" ON abonnements_push_vapid
    FOR ALL USING (auth.uid() = user_id);
```

ORM à ajouter :

```python
class AbonnementPushVapid(Base):
    __tablename__ = "abonnements_push_vapid"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[uuid.UUID] = mapped_column(ForeignKey("auth.users.id"))
    endpoint: Mapped[str] = mapped_column(unique=True)
    p256dh: Mapped[str]
    auth_key: Mapped[str]
    user_agent: Mapped[str | None]
    actif: Mapped[bool] = mapped_column(default=True)
    cree_le: Mapped[datetime] = mapped_column(default=func.now())
    dernier_ping: Mapped[datetime | None]
```

Changement `notif_web_core.py` :

- Remplacer `self._subscriptions: dict[str, dict]` par un accès DB via session SQLAlchemy
- `subscribe()` → `INSERT INTO abonnements_push_vapid` (upsert sur `endpoint`)
- `unsubscribe()` → `UPDATE actif = FALSE`
- `_send_all()` → `SELECT * WHERE actif = TRUE`

P-02 — Endpoint VAPID public key *(XS)* {#p-02--endpoint-vapid-public-key-*xs* }

Bug : B-02 — Aucun `GET /api/v1/push/vapid-public-key`

Fichier à modifier : `src/api/routes/push.py`

```python
@router.get("/vapid-public-key")
async def obtenir_cle_vapid_publique():
    """Retourne la clé publique VAPID pour l'abonnement Web Push côté frontend."""
    return {"public_key": obtenir_parametres().VAPID_PUBLIC_KEY}
```

P-03 — Scheduler ntfy digest + rappel courses *(XS)* {#p-03--scheduler-ntfy-digest--rappel-courses-*xs* }

Bug : B-03 — `envoyer_digest_quotidien()` et `envoyer_rappel_courses()` jamais schedulés

Fichier à modifier : `src/services/core/cron/jobs.py`

Ajouter dans la fonction d'initialisation APScheduler :

```
scheduler.add_job(
    ServiceNtfy.envoyer_digest_quotidien,
    trigger=CronTrigger(hour=9, minute=0),
    id="digest_ntfy",
    replace_existing=True,
)
scheduler.add_job(
    ServiceNtfy.envoyer_rappel_courses,
    trigger=CronTrigger(hour=18, minute=0),
    id="rappel_courses",
    replace_existing=True,
)
```

P-04 — Supprimer table `liste_cours` (doublon SQL) *(XS)* {#p-04--supprimer-table-liste_cours-doublon-sql-*xs* }

Bug : SQL-01 — doublon `liste_cours` / `listes_courses`

Fichier à modifier : `sql/INIT_COMPLET.sql`

- Supprimer le bloc `CREATE TABLE liste_cours`
- Vérifier et remplacer tout `INSERT INTO liste_cours` par `INSERT INTO listes_courses`
- Vérifier les `REFERENCES liste_cours` dans les FK

P-05 — Déplacer inline ALTER TABLE en tête *(S)* {#p-05--déplacer-inline-alter-table-en-tête-*s* }

Problème : SQL-05 — colonnes ajoutées en bas du fichier par `ALTER TABLE`

Fichier à modifier : `sql/INIT_COMPLET.sql`

- Chercher tous les `ALTER TABLE xxx ADD COLUMN` en bas du fichier
- Intégrer chaque `ADD COLUMN` directement dans le `CREATE TABLE` correspondant
- Supprimer les blocs `ALTER TABLE` une fois les colonnes intégrées

P-06 — Absorber `sql/migrations/001|002|003.sql` *(S)* {#p-06--absorber-sqlmigrations001002003sql-s }

Problème : SQL-05/SQL-06 — migrations SQL flottantes non intégrées

Actions :

1. Ouvrir `sql/migrations/001_routine_moment_journee.sql` → intégrer `moment_journee VARCHAR(20)` et `jour_semaine INTEGER` dans `CREATE TABLE routines`
2. Ouvrir `sql/migrations/002_standardize_user_id_uuid.sql` → corriger types `user_id VARCHAR → UUID` dans les 5 tables concernées (`preferences_utilisateurs` , `historique_actions` , `etats_persistants` , `configs_calendriers_externes` , `retours_recettes`)
3. Ouvrir `sql/migrations/003_add_cotes_historique.sql` → ajouter bloc `CREATE TABLE jeux_cotes_historique` dans la section Jeux
4. **Supprimer** les 3 fichiers `sql/migrations/001*.sql` , `002*.sql` , `003*.sql`

P-07 — Absorber `alembic/versions/` + archiver Alembic *(S)* {#p-07--absorber-alembicversions--archiver-alembic-s }

Problème : Alembic coexiste avec le workflow INIT_COMPLET — confusion

Actions :

1. `alembic/versions/a1b2c3d4e5f6_initial_baseline.py` → baseline vide, **supprimer directement**
2. `alembic/versions/f7e8d9c0b1a2_famille_refonte_phase1.py` → intégrer dans INIT_COMPLET :
   - `heure_debut TIME` dans `activites_famille`
   - `derniere_completion DATE` dans `routines`
   - `pour_qui` , `a_revendre` , `prix_revente_estime` , `vendu_le` dans `achats_famille`
   - 8 colonnes JSONB + `user_id UUID` dans `preferences_utilisateurs`
3. `alembic/versions/c8d1e2f3a4b5_anniversaire_checklists.py` → intégrer dans INIT_COMPLET :
   - Tables `checklists_anniversaire` + `items_checklist_anniversaire` + 7 index
4. **Supprimer** les 3 fichiers `alembic/versions/`
5. **Archiver** `alembic.ini` et `alembic/` (ou supprimer si Alembic n'est plus utilisé)

Validation après absorption — vérifier dans INIT_COMPLET que :

- [ ] `routines` : `moment_journee` , `jour_semaine` , `derniere_completion` présents
- [ ] `activites_famille` : `heure_debut TIME` présent
- [ ] `achats_famille` : 4 nouvelles colonnes présentes

- [ ] `preferences_utilisateurs` : `user_id UUID` + 8 colonnes JSONB présentes
- [ ] `jeux_cotes_historique` : table complète + 3 index + RLS
- [ ] `checklists_anniversaire` + `items_checklist_anniversaire` : 2 tables + 7 index

Sprint 2 — Bugs hauts + Nettoyage documentation

> **Durée** : ~4 jours | **Priorité** : 🟠 HAUT

Objectif

Corriger les bugs qui dégradent l'expérience utilisateur et assainir la documentation pour qu'ANALYSE_COMPLETE.md soit la référence unique.

B-06 — `url_source` absent du modèle ORM Recette *(S)* {#b-06--url_source-absent-du-modèle-orm-recette-s }

Fichiers à modifier :

- `src/core/models/recettes.py` — ajouter `url_source: Mapped[str | None]`
- `sql/INIT_COMPLET.sql` — ajouter la colonne dans `CREATE TABLE recettes`

B-07 — `verifier_saison` silencieusement ignoré *(S)* {#b-07--verifier_saison-silencieusement-ignoré-s }

Fichier à modifier : `src/services/core/cron/jobs.py`

- Remplacer le `logger.debug()` par `logger.warning()` si `hasattr()` est False
- Ajouter un mécanisme de fallback ou une alerte visible en startup

B-11 — Lien sidebar "Calendriers" → 404 *(S)* {#b-11--lien-sidebar-calendriers--404-*s* }

Fichier à modifier : `frontend/src/composants/disposition/barre-laterale.tsx`

- Supprimer ou corriger le lien vers `/famille/calendriers`
- Vérifier tous les liens de navigation pointant vers des pages inexistantes

B-12 — `RetourRecette.est_favori` vs `feedback` *(S)* {#b-12--retourrecetteest_favori-vs-feedback-*s* }

Fichiers à modifier :

- `frontend/src/types/recettes.ts` — aligner sur `feedback: "like" | "neutral" | "dislike"` OU
- `src/core/models/recettes.py` — ajouter un champ `est_favori: bool` calculé

CT-15 — Supprimer `STATUS_PHASES.md` *(S)* {#ct-15--supprimer-status_phasesmd-*s* }

Actions :

1. Supprimer `STATUS_PHASES.md` (1004 lignes redondantes)
2. Modifier `ROADMAP.md` : retirer la référence à `STATUS_PHASES.md`
3. Modifier `docs/INDEX.md` : retirer le lien, ajouter `ANALYSE_COMPLETE.md` comme référence principale

CT-16 — Nettoyer `ROADMAP.md` *(M)* {#ct-16--nettoyer-roadmapmd-*m* }

Fichier à modifier : `ROADMAP.md`

- Supprimer les sections d'historique des sprints déjà livrés
- Supprimer la section "Système de Phases A-AC" (absorbée)
- Aligner la table "Priorités 2 semaines" sur la section 13 de ANALYSE_COMPLETE.md
- Ajouter un lien vers `PLANNING_IMPLEMENTATION.md` pour le détail des sprints

CT-17 — Nettoyage `docs/` *(S)* {#ct-17--nettoyage-docs-*s* }

Actions :

1. Supprimer `docs/JEUX_AMELIORATIONS_PLAN.md` (jeux 100% complet)
2. Supprimer `docs/JEUX_PLAN_VALIDATION.md` (obsolète)
3. Supprimer `docs/guides/batch_cooking.md` (fichier vide 0 ligne)
4. Supprimer `docs/markdown-preview.md` (fichier de debug)
5. Mettre à jour `docs/INDEX.md` : retirer ces 4 liens, ajouter ANALYSE_COMPLETE.md

CT-11 — Mettre à jour la doc SQL *(S)* {#ct-11--mettre-à-jour-la-doc-sql-*s* }

Fichiers à modifier :

- `docs/MIGRATION_GUIDE.md` — retirer workflow SQL-file, documenter "INIT_COMPLET only"
- Archiver `sql/migrations/README.md` (ou supprimer)
- Archiver `alembic/README.md` (ou supprimer)

Sprint 3 — Tests manquants

> **Durée** : ~5 jours | **Priorité** : 🟠 IMPORTANT

Objectif

Couvrir les zones de test absentes pour éviter les régressions lors des prochains sprints.

CT-12 — Test cohérence schéma ORM ↔ SQL *(S)* {#ct-12--test-cohérence-schéma-orm--sql-*s* }

Fichier à créer : `tests/sql/test_schema_coherence.py`

Logique :

```python
def test_chaque_orm_a_un_create_table():
    """Vérifie que chaque __tablename__ ORM a un CREATE TABLE dans INIT_COMPLET.sql."""
    import re
    from pathlib import Path
    from src.core.models import Base  # importer tous les modèles

    sql_content = Path("sql/INIT_COMPLET.sql").read_text()
    sql_tables = set(re.findall(r"CREATE TABLE (?:IF NOT EXISTS )?(\w+)", sql_content))

    for mapper in Base.registry.mappers:
        table_name = mapper.class_.__tablename__
        assert table_name in sql_tables, f"Table ORM '{table_name}' absente de INIT_COMPLET.sql'
```

CT-13 — Tests push notifications *(M)* {#ct-13--tests-push-notifications-*m* }

Fichier à créer : `tests/api/test_push_notifications.py`

Tests à couvrir :

- `test_subscribe_persisted_in_db` — vérifier que POST /push/subscribe insère en DB (B-01)
- `test_subscribe_survives_restart` — vérifier que les abonnements sont rechargés depuis DB
- `test_vapid_public_key_endpoint` — GET /push/vapid-public-key retourne une clé (B-02)
- `test_ntfy_digest_scheduled` — vérifier que `digest_ntfy` est dans APScheduler (B-03)
- `test_rappel_courses_scheduled` — vérifier que `rappel_courses` est dans APScheduler (B-03)
- `test_unsubscribe_sets_actif_false` — désabonnement logique

CT-14 — Tests routes admin + RGPD *(M)* {#ct-14--tests-routes-admin--rgpd-*m* }

Fichier à créer : `tests/api/test_admin_routes.py`

Tests à couvrir :

- `test_lister_users_admin_only` — GET /admin/users exige rôle admin
- `test_lister_jobs_admin_only` — GET /admin/jobs exige rôle admin
- `test_executer_job_admin_only` — POST /admin/jobs/{id}/run exige rôle admin

- `test_export_rgpd_data` — GET /admin/export-data retourne les données utilisateur
- `test_delete_account_rgpd` — DELETE /admin/delete-account supprime les données

CT-07 — Tests famille (achats + garde) *(M)* {#ct-07--tests-famille-achats--garde-*m* }

Fichiers à créer :

- `tests/api/test_famille_achats.py`
  - CRUD achats famille
  - Suggestions IA enrichies (mock Mistral)
  - Scoring IA achat
  - Suggestions Vinted
- `tests/api/test_famille_garde.py`
  - Configuration zones garde
  - Semaines fermeture crèche
  - Jours-sans-crèche intégrés au planning

Sprint 4 — Features Cuisine/Famille/Jules

> **Durée** : ~5 jours | **Priorité** : ⭐ ⭐ ⭐ HAUTE VALEUR

Objectif

Implémenter les features les plus demandées pour Jules et le lien cuisine ↔ famille.

CT-09 — Bouton Version Jules + profil aliments exclus *(S)* {#ct-09--bouton-version-jules--profil-aliments-exclus-*s* }

Backend — `src/services/famille/version_recette_jules.py` (nouveau)

```
class ServiceVersionRecetteJules:
    """Génère une VersionRecette adaptée à Jules via Mistral."""

    PROMPT_SYSTEM = """Tu es spécialiste nutrition pédiatrique.
    Adapte la recette pour un enfant de {age_mois} mois.
    Aliments exclus pour cet enfant : {aliments_exclus}
    Règles absolues : supprimer sel ajouté, alcool → fond de volaille,
    saumon fumé → saumon cuit, viande/poisson cru → cuisson complète,
    épices fortes → supprimer ou remplacer par herbes douces."""

    def generer_version_jules(self, recette_id: int, profil_jules: dict) -> VersionRecette:
        ...
```

Backend — `src/api/routes/recettes.py`

Ajouter endpoint :

```
@router.post("/{recette_id}/version-jules", response_model=VersionRecetteResponse)
async def generer_version_jules(recette_id: int, user: dict = Depends(require_auth)):
    ...
```

Backend — `src/api/routes/famille.py`

Ajouter endpoint profil aliments exclus :

```
@router.get("/jules/aliments-exclus")
@router.put("/jules/aliments-exclus")
```

Frontend — `frontend/src/app/(app)/cuisine/recettes/[id]/page.tsx`

- Ajouter bouton 🍢 "Version Jules"
- On-click → `POST /api/v1/recettes/{id}/version-jules`
- Afficher la version adaptée inline ou dans un onglet

Frontend — `frontend/src/app/(app)/famille/jules/page.tsx`

- Ajouter section "Aliments exclus" : liste configurable (chips/tags + input ajout)
- Sauvegarde via `PUT /api/v1/famille/jules/aliments-exclus`

CT-06 — Génération recette depuis photo plat (Pixtral) *(S)* {#ct-06--génération-recette-depuis-photo-plat-pixtral-*s* }

Fichier à modifier : `src/services/integrations/multimodal.py`

Ajouter méthode :

```
async def generer_recette_depuis_photo(image_base64: str) -> dict:
    """Pixtral identifie le plat → Mistral génère la recette complète."""
    # 1. Pixtral : identifier le plat, ingrédients visibles, technique de cuisson
    # 2. Mistral : générer RecetteCreate complète (nom, ingrédients, étapes, temps, calories)
    # 3. Retourner le dict compatible RecetteCreate
```

Nouveau endpoint : `src/api/routes/recettes.py`

```
@router.post("/generer-depuis-photo", response_model=RecetteResponse)
async def generer_depuis_photo(image: UploadFile, user: dict = Depends(require_auth)):
    ...
```

Frontend : Ajouter bouton "📷 Générer depuis une photo" dans `/cuisine/recettes/nouvelle`

CT-05 — Coaching hebdo Jules (IA-06) *(S)* {#ct-05--coaching-hebdo-jules-ia-06-*s* }

Fichier à modifier : `src/services/famille/jules_ai.py`

Ajouter méthode `generer_coaching_hebdo()` :

- Agréger : activités de la semaine, repas planifiés avec version Jules, derniers jalons, dates vaccins
- Générer un message Mistral contextuel
- Retourner un `MessageCoachingJules` avec conseils + alertes

Nouveau endpoint : `GET /api/v1/famille/jules/coaching-hebdo`

Frontend : Widget "Coaching Jules" dans `/famille/jules` (affiché si disponible)

Sprint 5 — Notifications email + Admin étendu

> **Durée** : ~8 jours | **Priorité** : 🟠 IMPORTANT

Objectif

Ajouter le canal email (manquant total) et créer le panneau admin de déclenchement manuel.

CT-01 — Canal email avec Resend *(L)* {#ct-01--canal-email-avec-resend-/ }

Dépendance

`pip install resend` + variable `RESEND_API_KEY` dans `.env.local`

Fichiers à créer

`src/services/core/notifications/notif_email.py` :

```
class ServiceEmail:
    """Envoi d'emails transactionnels via Resend."""

    def envoyer_reset_password(self, email: str, token: str) -> bool: ...
    def envoyer_verification_email(self, email: str, token: str) -> bool: ...
    def envoyer_resume_hebdo(self, email: str, resume: dict) -> bool: ...
    def envoyer_rapport_mensuel(self, email: str, rapport: dict) -> bool: ...
    def envoyer_alerte_critique(self, email: str, alerte: dict) -> bool: ...
    def envoyer_invitation_famille(self, email: str, invitant: str) -> bool: ...
```

`src/services/core/notifications/notif_dispatcher.py` :

```python
class DispatcherNotifications:
    """Dispatcher multi-canal : email / push / ntfy / whatsapp."""

    def envoyer(
        self,
        user_id: str,
        message: str,
        canaux: list[str] = ["ntfy", "push"],
        **kwargs
    ) -> dict[str, bool]:
        """Retourne {canal: succès} pour chaque canal demandé."""
```

Intégration `src/api/routes/auth.py` :

- `POST /auth/forgot-password` → `ServiceEmail.envoyer_reset_password()`
- `POST /auth/verify-email` → `ServiceEmail.envoyer_verification_email()`

CT-02 — Mode Admin étendu *(M)* {#ct-02--mode-admin-étendu-*m* }

Backend — `src/api/routes/admin.py` (enrichi)

Ajouter endpoints :

```python
# Liste tous les jobs et leur statut
@router.get("/jobs", response_model=list[JobInfoResponse])
async def lister_jobs(user: dict = Depends(require_role("admin"))): ...

# Déclenche manuellement un job
@router.post("/jobs/{job_id}/run")
async def executer_job(job_id: str, user: dict = Depends(require_role("admin"))): ...

# Envoie une notif test
@router.post("/notifications/test")
async def envoyer_notification_test(
    canal: Literal["ntfy", "push", "email"],
    message: str,
    user: dict = Depends(require_role("admin"))
): ...

# Purge le cache
@router.post("/cache/purge")
async def purger_cache(
    pattern: str = "*",
    user: dict = Depends(require_role("admin"))
): ...

# Liste les utilisateurs
@router.get("/users", response_model=list[UtilisateurAdminResponse])
async def lister_utilisateurs(user: dict = Depends(require_role("admin"))): ...
```

Jobs disponibles (à mapper dans `jobs_disponibles` dict) :

- `rappels_famille` , `rappels_maison` , `rappels_generaux`
- `push_quotidien` , `digest_ntfy` , `rappel_courses`
- `entretien_saisonnier` , `enrichissement_catalogues`
- `resume_hebdo` , `peremptions_urgentes` , `score_bienetre`

Frontend — `frontend/src/app/(app)/admin/` (nouveaux fichiers)

`jobs/page.tsx` :

- Table des jobs avec : ID, libellé, schedule, dernier run, statut (✅ / ❌)
- Bouton ▶ "Exécuter maintenant" par ligne
- Toast confirmation + feedback statut

Mise à jour `page.tsx` (admin principal) :

- Ajouter onglets : Jobs | Notifications test | Cache | Utilisateurs
- Onglet "Notifications test" : sélecteur canal + champ message + bouton envoyer
- Onglet "Cache" : afficher stats hits/misses, bouton purge par module
- Onglet "Utilisateurs" : liste comptes, modification rôle, désactivation

Sprint 6 — Cron jobs + SQL avancé

> **Durée** : ~5 jours | **Priorité** : ⭐ ⭐ MOYEN

CT-03 — Job J-02 : Push contextuel soir 18h *(M)* {#ct-03--job-j-02--push-contextuel-soir-18h-*m* }

Fichier à modifier : `src/services/core/cron/jobs.py`

Nouveau job `_job_push_contextuel_soir()` :

1. Charger le planning du lendemain (`GET /api/v1/planning/semaine`)
2. Charger la météo du lendemain (Open-Meteo API)
3. Charger les produits à décongeler (si plat avec viande/poisson)
4. Générer un message Mistral contextuel court
5. Envoyer via `DispatcherNotifications` sur ntfy + Web Push

Ajouter au scheduler APScheduler :

```
scheduler.add_job(_job_push_contextuel_soir, CronTrigger(hour=18, minute=0), id="push_contextuel
```

CT-04 — Job J-01 : Résumé hebdo lundi 7h30 *(M)* {#ct-04--job-j-01--résumé-hebdo-lundi-7h30-*m* }

Fichier à modifier : `src/services/famille/service_resume_hebdo.py` (ou créer)

`ServiceResumeHebdo.generer_resume_semaine()` :

1. Agréger les données de la semaine passée :

- Recettes cuisinées (depuis `repas` avec `date_repas` de la semaine)
- Budget dépensé (depuis `articles_budget` de la semaine)
- Activités Jules (depuis `activites_famille` de la semaine)
- Tâches maison faites (depuis `taches_entretien` de la semaine)
- Gains/pertes jeux (depuis `jeux_paris_sportifs` de la semaine)

2. Mistral génère un résumé narratif court
3. Envoyer via `DispatcherNotifications` (ntfy + email si configuré)

Ajouter au scheduler :

```
scheduler.add_job(_job_resume_hebdo, CronTrigger(day_of_week="mon", hour=7, minute=30), id="resu
```

CT-08 — Index manquants sur relations fréquentes *(S)* {#ct-08--index-manquants-sur-relations-fréquentes-*s* }

Fichier à modifier : `sql/INIT_COMPLET.sql`

Ajouter dans la section des `INDEX` :

```
CREATE INDEX IF NOT EXISTS idx_recette_ingredients_ingredient_id ON recette_ingredients(ingredie
CREATE INDEX IF NOT EXISTS idx_repas_planning_id ON repas(planning_id);
CREATE INDEX IF NOT EXISTS idx_articles_courses_liste_id ON articles_courses(liste_id);
CREATE INDEX IF NOT EXISTS idx_historique_inventaire_article_id ON historique_inventaire(article
CREATE INDEX IF NOT EXISTS idx_jalons_profil_id ON jalons(profil_enfant_id);
CREATE INDEX IF NOT EXISTS idx_paris_user_date ON jeux_paris_sportifs(user_id, date_pari);
```

CT-10 — Audit tables orphelines ORM ↔ SQL *(M)* {#ct-10--audit-tables-orphelines-orm--sql-*m* }

Actions :

1. Lister toutes les `__tablename__` dans `src/core/models/voyage.py` — vérifier si elles ont un `CREATE TABLE` dans INIT_COMPLET.sql
2. Vérifier `journal_bord`, `sessions_travail`, `log_statut_objets` — ont-ils un ORM ou sont-ils orphelins SQL ?
3. Pour chaque table orpheline SQL (sans ORM) : soit créer le modèle ORM, soit supprimer la table

4. Pour chaque modèle ORM sans SQL (ex: `album.py` si toujours présent) : soit ajouter le CREATE TABLE, soit supprimer le modèle

Sprint 7 — Inter-modules + Dashboard

> **Durée** : ~2 semaines | **Priorité** : ⭐ ⭐ ⭐ HAUTE VALEUR

MT-01 — Cellier ↔ Inventaire cuisine *(M)* {#mt-01--cellier--inventaire-cuisine-*m* }

Backend — `src/api/routes/inventaire.py` :
Ajouter endpoint :

```
@router.get("/consolide", response_model=list[ArticleConsolideResponse])
async def inventaire_consolide(user: dict = Depends(require_auth)):
    """Merge ArticleInventaire (cuisine) + ArticleCellier (maison) en vue unifiée."""
```

Logique :

- Champ `source: "cuisine" | "cellier"` dans `ArticleConsolideResponse`
- Merge par nom normalisé (Levenshtein ou correspondance exacte)
- Générer la liste de courses depuis la vue consolidée

MT-03 — Score bien-être global (IA-09) *(M)* {#mt-03--score-bien-être-global-ia-09-*m* }

Backend — `src/services/dashboard/score_bienetre.py` (nouveau) :

```
Score = diversité_alimentaire_score (40%) + nutriscore_moyen_score (30%) + activites_sportives_s
```

Calcul :

- `diversité_alimentaire` : nb de familles d'aliments différentes dans les repas planifiés / 7 jours
- `nutriscore_moyen` : moyenne des Nutri-Scores des recettes planifiées (A=5, B=4, C=3, D=2, E=1)
- `activites_sportives` : nb d'activités avec type "sport/plein-air" dans la semaine

Endpoint : `GET /api/v1/dashboard/score-bienetre`

Frontend : Widget "Score bien-être" dans le dashboard (jauge circulaire 0-100 + trend semaine précédente)

MT-06 — Widgets dashboard configurables *(L)* {#mt-06--widgets-dashboard-configurables-*l* }

Backend : `GET /api/v1/dashboard/config` + `PUT /api/v1/dashboard/config`

- Stocker la config widgets dans `preferences_utilisateurs.config_dashboard` (JSONB)

Frontend :

- Mode "Configurer le dashboard" activable
- Drag & drop des widgets (react-beautiful-dnd ou dnd-kit)
- Choix des métriques affichées par widget
- Sauvegarde de la config personnalisée

MT-08 — Timeline vie familiale (Innovation-02) *(M)* {#mt-08--timeline-vie-familiale-innovation-02-*m* }

Backend — `GET /api/v1/famille/timeline` :
Sources à agréger :

- Jalons Jules (`jalons`)
- Événements familiaux (`evenements_familiaux`)
- Projets maison complétés (`projets_maison` avec `statut = "terminé"`)
- Matchs mémorables (paris > X% ROI ou résultat notable)

Frontend — `frontend/src/app/(app)/famille/timeline/page.tsx` :

- Visualisation chronologique verticale (CSS scroll)
- Filtrable par catégorie (Jules / Maison / Famille / Jeux)
- Export PDF de la timeline

MT-09 — OCR photo-frigo → auto-sync inventaire *(S)* {#mt-09--ocr-photo-frigo--auto-sync-inventaire-*s* }

Fichier à modifier : `frontend/src/app/(app)/cuisine/inventaire/page.tsx`

- Sur le résultat OCR photo-frigo, ajouter bouton "Tout ajouter à l'inventaire"
- `POST /api/v1/inventaire/bulk` avec les articles reconnus
- Option checkbox par article pour sélection avant import

Backend : Vérifier/créer `POST /api/v1/inventaire/bulk` (import en lot)

Sprint 8 — Canal WhatsApp + IA avancée

> **Durée** : ~2 semaines | **Priorité** : ⭐ ⭐ MOYEN

MT-02 — Canal WhatsApp + webhook entrant *(L)* {#mt-02--canal-whatsapp--webhook-entrant-*l* }

Dépendance : Compte Twilio WhatsApp Business ou Meta Cloud API

`src/services/core/notifications/notif_whatsapp.py` :

```python
class ServiceWhatsApp:
    def envoyer_message(self, telephone: str, message: str) -> bool: ...
    def envoyer_liste_courses(self, telephone: str, articles: list) -> bool: ...
    def envoyer_resume_hebdo(self, telephone: str, resume: dict) -> bool: ...
```

`src/api/routes/webhooks.py` — ajouter route :

```python
@router.post("/whatsapp")
async def webhook_whatsapp(request: Request):
    """Webhook Twilio → extraire intention → dispatcher vers API interne."""
    # 1. Extraire message texte de la requête Twilio (valider signature HMAC)
    # 2. Mistral : interpréter intention
    # 3. Dispatcher vers API interne
    # 4. Répondre TwiML avec confirmation
```

Sécurité : Valider la signature HMAC Twilio sur chaque webhook entrant.

MT-04 — Alertes météo contextuelles cross-modules *(M)* {#mt-04--alertes-météo-contextuelles-cross-modules-*m* }

Backend — `GET /api/v1/dashboard/alertes-contextuelles` :

- Charger météo Open-Meteo pour les 48 prochaines heures
- Évaluer les conditions (gel, canicule, vent fort, pluie, pollen)
- Générer une liste d'alertes contextuelles cross-modules selon le tableau section 7

Frontend — Widget "Alertes contextuelles" dans le dashboard :

- Afficher les alertes actives (max 3 simultanées)
- Chaque alerte avec icône, module concerné, action suggérée

MT-07 — Assistant vocal (IA-01) *(L)* {#mt-07--assistant-vocal-ia-01-*l* }

Frontend :

- Bouton microphone flottant dans l'app (accessible depuis toutes les pages)
- Web Speech API → transcription
- `POST /api/v1/assistant/commande-vocale` avec le texte transcrit

Backend — `src/api/routes/` **(nouveau endpoint)** :

- Mistral extrait l'intention et les paramètres
- Dispatch vers l'API interne correspondante
- Retourne une réponse textuelle confirmant l'action

Intentions à gérer :

- "Ajoute [article] à la liste de courses" → `POST /courses/{id}/articles`
- "Jules pèse [X] kg" → `POST /famille/jules/croissance`
- "Rappelle-moi [action] [quand]" → `POST /maison/routines`
- "Quel est mon planning de demain ?" → `GET /planning/semaine` + résumé oral

Sprint 9 — Innovations & Long terme

> **Durée** : 3+ mois | **Priorité** : ⭐ NICE-TO-HAVE

LT-01 — Intégration Garmin santé/sport (Innovation-05/06) *(L)* {#lt-01--intégration-garmin-santésport-innovation-0506-*l* }

- OAuth Garmin Connect
- Webhook daily sync (`ResumeQuotidienGarmin`)
- Calories brûlées → recommandations repas du soir
- Activités enregistrées → contribution score bien-être (IA-09)

LT-02 — Gamification sport + alimentation (Innovation-07) *(M)* {#lt-02--gamification-sport--alimentation-innovation-07-*m* }

- Activer `ServiceGamification` existant
- Points pour activités physiques + badges Nutri-Score hebdo
- Widget "Points famille" sur dashboard

LT-03 — Mode Voyage avec checklists intelligentes (Innovation-04) *(M)* {#lt-03--mode-voyage-avec-checklists-intelligentes-innovation-04-*m* }

- 4 types de séjour (Mer été, Montagne hiver/été, Mer hiver)
- Générer checklist + courses adaptées depuis template + IA
- Bouton "Acheté sur place" + apprentissage à la fin du séjour
- Suspension automatique des rappels arrosage + planning cuisine allégé

LT-04 — Automations "Si → Alors" (Innovation-01) *(L)* {#lt-04--automations-si--alors-innovation-01-*l* }

- Table `automations` en DB + moteur de règles simplifié
- Interface visuelle déclencheur → action
- Exemples : stock lait < 2 → ajouter, pari gagné > 50€ → budget loisirs

Quick Wins — À intercaler entre les sprints

> Chacun ≤ 2h, intégrer au sprint le plus proche thématiquement

QW	Feature	Sprint cible
QW-01	Widget météo sur dashboard	Sprint 7 (avec MT-03 dashboard)
QW-02	Bouton "Recette Surprise" filtré saison + frigo	Sprint 4 (avec CT-06)
QW-03	Partage liste courses par QR code	Sprint 5 (avec CT-01)
QW-06	"Aujourd'hui dans l'histoire de la famille"	Sprint 7 (avec MT-08)
QW-08	Impression optimisée recette (CSS print)	Sprint 2 (nettoyage)
QW-04	Compteur jours depuis dernière cuisine	Sprint 4
QW-07	Mode sombre/clair auto après 21h	Sprint 2
QW-10	Score anti-gaspi partageable (image)	Sprint 6

Suivi d'avancement

Sprint 1 — Bugs critiques + SQL fondations

- [ ] P-01 — Persistance push subscriptions en DB
- [ ] P-02 — Endpoint VAPID public key
- [ ] P-03 — Scheduler ntfy digest + rappel courses
- [ ] P-04 — Supprimer table `liste_cours`
- [ ] P-05 — Déplacer inline ALTER TABLE en tête
- [ ] P-06 — Absorber sql/migrations/001|002|003
- [ ] P-07 — Absorber alembic/versions/ + archiver Alembic

Sprint 2 — Bugs hauts + Nettoyage doc

- [ ] B-06 — `url_source` absent du modèle Recette
- [ ] B-07 — `verifier_saison` silencieux
- [ ] B-11 — Lien sidebar Calendriers → 404
- [ ] B-12 — `est_favori` vs `feedback` TypeScript

- [ ] CT-15 — Supprimer STATUS_PHASES.md
- [ ] CT-16 — Nettoyer ROADMAP.md
- [ ] CT-17 — Nettoyage docs/
- [ ] CT-11 — Mettre à jour doc SQL

Sprint 3 — Tests

- [ ] CT-12 — Test cohérence schéma ORM ↔ SQL
- [ ] CT-13 — Tests push notifications
- [ ] CT-14 — Tests routes admin + RGPD
- [ ] CT-07 — Tests famille achats + garde

Sprint 4 — Features cuisine/famille/Jules

- [ ] CT-09 — Bouton Version Jules + profil aliments exclus
- [ ] CT-06 — Génération recette depuis photo
- [ ] CT-05 — Coaching hebdo Jules (IA-06)
- [ ] QW-02 — Recette Surprise
- [ ] QW-04 — Compteur jours depuis dernière cuisine

Sprint 5 — Notifications email + Admin étendu

- [ ] CT-01 — Canal email (Resend)
- [ ] CT-02 — Mode Admin étendu (jobs + users + cache)
- [ ] QW-03 — Partage liste courses QR

Sprint 6 — Cron jobs + SQL avancé

- [ ] CT-03 — Job J-02 push contextuel soir
- [ ] CT-04 — Job J-01 résumé hebdo
- [ ] CT-08 — Index manquants SQL
- [ ] CT-10 — Audit tables orphelines ORM ↔ SQL
- [ ] QW-10 — Score anti-gaspi partageable

Sprint 7 — Inter-modules + Dashboard

- [ ] MT-01 — Cellier ↔ Inventaire cuisine
- [ ] MT-03 — Score bien-être (IA-09)
- [ ] MT-06 — Widgets dashboard configurables

- ☐ MT-08 — Timeline vie familiale
- ☐ MT-09 — OCR photo-frigo auto-sync
- ☐ QW-01 — Widget météo dashboard
- ☐ QW-06 — Aujourd'hui dans l'histoire

Sprint 8 — WhatsApp + IA avancée

- ☐ MT-02 — Canal WhatsApp
- ☐ MT-04 — Alertes météo contextuelles
- ☐ MT-07 — Assistant vocal

Sprint 9 — Long terme

- ☐ LT-01 — Garmin santé/sport
- ☐ LT-02 — Gamification sport + alimentation
- ☐ LT-03 — Mode Voyage
- ☐ LT-04 — Automations Si → Alors

Dépendances entre items

```
P-01 (push DB) ──────────────────→ CT-13 (tests push)
P-02 (VAPID endpoint) ──────────→ CT-13
P-03 (scheduler ntfy) ──────────→ CT-13
P-06 (absorber migrations SQL) ──→ CT-12 (test cohérence)
P-07 (absorber alembic) ────────→ CT-12
CT-01 (email service) ──────────→ CT-04 (résumé hebdo email)
CT-01 ──────────────────────────→ J-07 (rapport mensuel email)
CT-02 (admin panel) ────────────→ déclenchement CT-03, CT-04
CT-09 (version Jules) ──────────→ MT-04 (alertes contextuelles Jules)
MT-01 (cellier ↔ inventaire) ───→ courses list consolidée
MT-03 (score bien-être) ────────→ MT-06 (widget dashboard)
```

Plan généré le 28 mars 2026 — GitHub Copilot

Référence source : `ANALYSE_COMPLETE.md`